

重み無しグラフによる単一始点最長経路探索の開発

～新潟県から目的地までの道をいろんな手法で探すコト!?!～

遠雷亭

皆さんは、可能な限り多くの県を経由して目的地までのんびり旅したいと思ったことはあるだろうか。

私は無い。

本稿は、新潟県と各都道府県を結ぶ最長経路の探索を複数手法で実装し、それらを評価したものである。

この記事を通して、皆さんの旅路が少しでも長くなれば幸いである。

--目次--

まえがき

経緯

条件設定

プログラムの説明

- 最長経路探索について
- スタックと深さ優先探索について
- グラフの作成
- 実用性の配慮

実行結果と考察

改良

1. 並列処理および並行処理
2. 集合 set で検索
3. 都道府県番号で管理
4. 言語の変更
5. <ヒューリスティック探索> 反復局所探索法
6. <ヒューリスティック探索> 焼きなまし法
7. 量子アニーリングと QUBO 行列の考察
8. ライブラリの利用

結論と展望

謝辞

参考文献

経緯

「けいい」と読むか、「いきさつ」と読むか、個人の判断によるところだろうが、私は「いきさつ」派である。さて、事の経緯を説明させていただきたい。

あまり長旅が好きでない私がこのプログラムの開発を始めた理由は、講義の課題だ。

プログラミングの課題で「スタックのデータ構造を用いてプログラムを python で作成せよ」という自由課題があった。おそらく入力した数字をスタックさせて新しい方から順に出力させる単純なプログラムでも充分だったのだろうが、せっかくなら実用できて面白そうなプログラムを作りたいという動機が今回の記事の発端である。

そのためこの記事は python を主な言語として話が展開される。技術選定において処理速度やライブラリの充実度、取り扱う問題の種類など様々な要件があるだろうが、python を選択した理由は、講義で指定されていた言語だからである。

また、動機はもう一点。新潟大学コンピュータクラブ設立 50 年目に刊行する記事である点、コミックマーケットに出展する点から、「技術っぽい話かつ新潟に興味を持っていた話」を題材にしたかったからという理由もある。

50 年目に恥じない内容であったか、新潟に興味を抱く内容であったかなど、ぜひ直接弊学へ訪れて感想を伝えていただきたい。

もちろんお越しの際は、最長経路で。

条件設定

目的

重み無しグラフを用いて、始点から目的地まで一筆書きで移動する場合の最長経路を探索する。

グラフ構造について

ノードは 47 都道府県とする。

エッジは隣接する都道府県間を接続するものとする。

隣接関係は参考文献[1]を基にした。

ここでは、地理的に接続していることよりも移動可能な経路で接続されていることを優先する場合がある。そのような隣接関係に関する特例は後述する。

経路制約

始点は新潟県とする。

接続しているノード間（都道府県間）のみ移動可能である。

一度通過したノードへの再訪問を禁止する（一筆書きの経路のみ選択できる）。

（※グラフ理論の起源には、「一筆書きできるか」という問題が関連しているらしい。

「ケーニヒスベルクの橋の問題」に対するオイラーの解法を参照していただきたい。）

プログラムの説明

ざっくりプログラム全体の構造を説明すると、

- ・入出力を管理するメインプログラム
- ・都道府県の隣接関係を表現したグラフ
- ・経路探索プログラム

である。経路探索→グラフ→メインプログラムの順に見ていこう。

--最長経路探索について

最短経路を求めることは、「目的地に早く着きたい」「早く荷物を運びたい」「ネットワークの通信効率を良くしたい」など社会にとって重要な動機が多い。

重み無しグラフにおいて最短経路を探索する場合には、幅優先探索が単純かつ有効である。また、重み付けグラフにおいてはダイクストラ法や A*法などがある。

（Google マップではマップを階層化することでさらに効率よく最短経路を探索するアルゴリズムが採用されている。これらの話題に興味を持った方は参考文献[2]（ゆるコンピュータ科学ラジオ|ダイクストラ法および最短経路回）を参照していただきたい。）

一方最長経路探索は、ダイクストラ法やそこから派生したアルゴリズムを流用することはできない。経路の重みを-1 とすることで最短経路探索のアルゴリズムを流用できるように見えるが、ダイクストラ法は負の値を扱うことができない。ベルマンフォード法は負の重みをもつ経路を扱えるが、最終的な重みの総和は正である必要があるため、すべての経路の重みを-1 とする場合うまくいかない。これらからもわかるように、今回の条件設定における最長経路探索は NP 完全な問題、あるいは NP 困難な問題であるらしい。要するに難しいってこと。そのため既存の華麗なアルゴリズムを採用するのではなく、単純な深さ優先探

索を軸としていろいろ工夫を加えるという方向でひとまず実装してみた。

最長経路探索は最短経路探索と比較すると動機が少ないように感じる。二つ動機をひねり出してみる。一つは観光業におけるルート設計だろうか。観光地や娯楽施設において、意図的に動線を延伸することで消費機会を増やすことで収益を増やす案である。しかし、たくさん歩かせるとユーザ体験が悪化するためあまり良い効果があると思えない。そしてもう一つの案が、旅行計画だ。日本中をのんびり見て回りたい旅人にとって、あるいは一度の旅で訪問する都道府県を最大にしたい効率厨にとって、このプログラムは長旅のお供となるのかもしれない。

--スタックと深さ優先探索について

まず、データを取り扱う構造について説明する。キューとは「先入先出し」「First In, First Out」とも呼ばれ、ラーメン屋やコミックマーケットにて形成される待機列のようなものだ。最後尾に並び、先頭から順に出ていく。

対をなすスタックは「後入れ先出し」「Last In, First Out」とも呼ばれる構造で、最後に追加された要素から順に取り出される。箱の中にものを積み上げ、上から順に取っていくイメージだ。

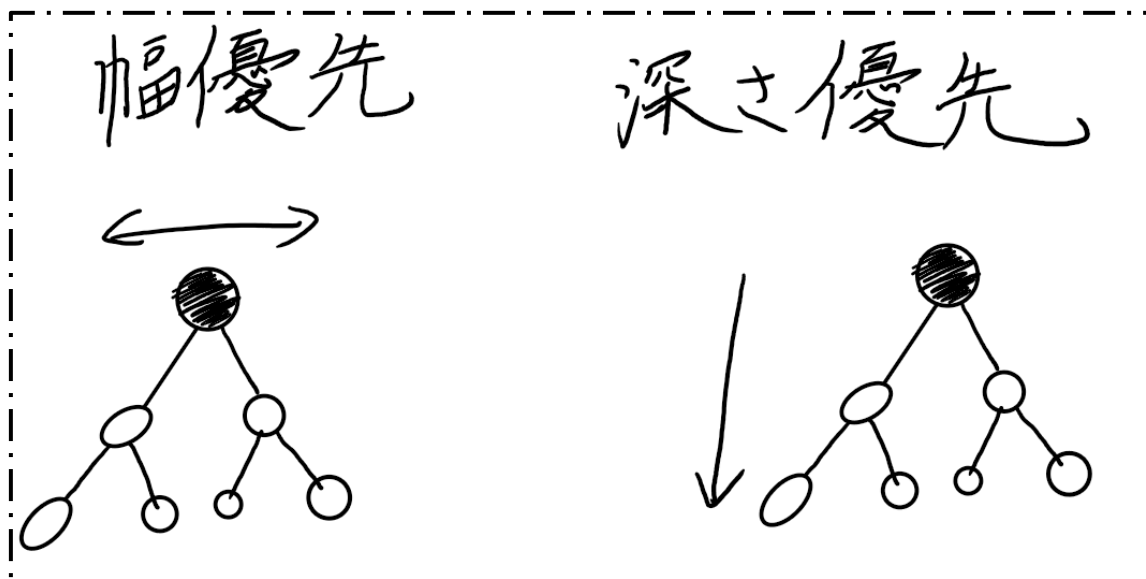
そんなキューとスタックであるが、幅優先探索と深さ優先探索にそれぞれ利用されるデータ構造である。

二分木のどこかに隠されたゴールを探すことを考えてみよう。

幅優先探索であればスタートのノードに近い階層から順に探索していく。だから最初にゴールをみつけたときの経路が最短経路であり、これを実装するデータ構造がキューである。

深さ優先探索は行き止まりまで進んでから一つ浅い階層へ戻り、また深い層へ進むという探索方法を取る。これを実装する際、「経路の終端を参照して一つ浅い階層へ戻る」という操作は「最新の要素を取り出して別の要素を入れる」という操作で言い換えることができ、スタックの構造が必要となる。

深さ優先探索を用いたとしても最長経路がすぐに見つかるわけではないが、一回目の探索から一番遠いノードまで訪問するため、幅優先探索と比べて早めに最長経路が見つかる場合が多い。どちらの探索方式でも、順序に従って最後まで探索を進めていけば全経路を探索できる点が重要である。



(図 1. 幅優先/深さ優先探索の理解を助けるわけでもないイメージ図)

-- グラフの作成

ここでは、都道府県の隣接関係を表現するため二分木ではなく循環を許すグラフを用いた。
都道府県隣接関係したグラフにおいての特筆事項をここで説明する。

- ・ 兵庫県-徳島県間など、橋でつながっている経路は隣接しているものとした
- ・ 隣接する都道府県へのみ移動できる都合上、北海道-青森県間、鹿児島県-沖縄県間は隣接しているものとした

(実際にこの経路を移動する方は、フェリーなどをご利用ください)

- ・ 福島県-群馬県間と長野県-埼玉県間の隣接関係を削除した
(徒歩以外の交通手段がないため、現実的な経路でないと判断した)

作成したグラフのソースコードの一部を以下に示す。

```

class PrefectureGraph:
    def __init__(self):
        # 都道府県同士の隣接関係を辞書で管理
        self.prefectures = {
            '北海道': ['青森県'],
            '青森県': ['岩手県', '秋田県', '北海道'],
            '岩手県': ['青森県', '宮城県', '秋田県'],
            '宮城県': ['岩手県', '秋田県', '山形県', '福島県'],
            '秋田県': ['青森県', '岩手県', '宮城県', '山形県'],
            '山形県': ['秋田県', '宮城県', '福島県', '新潟県'],
            '福島県': ['宮城県', '山形県', '新潟県', '茨城県', '栃木県'],
            ~~~~~(省略)
            '新潟県': ['山形県', '福島県', '群馬県', '長野県', '富山県'],
            ~~~~~(省略)
            '鹿児島県': ['熊本県', '宮崎県', '沖縄県'],
            '沖縄県': ['鹿児島県']
        }

```

（グラフ理論について勉強したい時期が私にはあったのだが、理論やイメージ図での説明ばかりで、実装する場合の書き方の説明が乏しかったように思う。もしグラフ理論に興味がある方、機械学習やネットワークに興味がある方のきっかけになれば嬉しい。）

-- 実用性の配慮

実用性、あるいはユーザビリティにも少しばかり配慮した。

前述したとおりこの問題は NP 完全な問題であるらしく、私は全経路探索以外の方法で最長経路を求めたことを証明する手法を見つけられなかった。全経路を探索するせいでプログラムの処理に時間を要するため、二つの配慮を施した。

進捗表示

処理に時間がかかるため、進捗状況をユーザに伝えるべきだと考えた。

そこで、暫定的な最長経路長が更新されるたびに出力するようにした。

（例）“さらに長い経路を発見：12 県通過”

目標経路長の入力

ユーザから目標とする経路長の入力を受け付け、その経路長を超えた場合に探索を終了

するようにした。これにより、最長でなくともユーザが納得する経路長を現実的な計算時間で探索することができ、実用性が向上すると考える。

最長経路を探索したい場合には、“47”と入力すればよい。

実行結果と考察

実行結果を以下に示す。

新潟県-福島県間の最長経路探索

542 秒(9 分)

新潟県からの最長経路探索プログラム

目的地の都道府県名を入力してください: 福島県

目標とする経路長を入力してください
この値を超えるように探索を繰り返します: 47

新潟県から福島県までの経路の探索を開始

参考: 最短経路長は1県
さらに長い経路を発見: 13県通過
さらに長い経路を発見: 19県通過
さらに長い経路を発見: 20県通過
さらに長い経路を発見: 21県通過
さらに長い経路を発見: 22県通過
さらに長い経路を発見: 23県通過
探索終了。513602647本の経路を調査
条件を満たす経路は見つかりませんでした。
23県を通過する経路が最長経路です
経路:
新潟県 -> 富山県 -> 長野県 -> 岐阜県 -> 石川県 -> 福井県 -> 滋賀県 -> 京都府 -> 兵庫県 -> 大阪府 -> 和歌山県 -> 奈良県 -> 三重県 -> 愛知県 -> 静岡県 -> 山梨県 -> 神奈川県 -> 東京都 -> 埼玉県 -> 千葉県 -> 茨城県 -> 群馬県 -> 栃木県 -> 福島県
探索時間: 542.20秒
終了

新潟県-群馬県間の最長経路探索

2368 秒(40 分)

※最長経路長を示す出力に誤りがある。

誤: 9 県 正: 28 県

この値を超えるように探索を繰り返します： 45

新潟県から群馬県までの経路の探索を開始

参考：最短経路は1県を通過する経路
さらに長い経路を発見： 11県通過
さらに長い経路を発見： 12県通過
さらに長い経路を発見： 13県通過
さらに長い経路を発見： 17県通過
さらに長い経路を発見： 18県通過
さらに長い経路を発見： 19県通過
さらに長い経路を発見： 20県通過
さらに長い経路を発見： 21県通過
さらに長い経路を発見： 22県通過
さらに長い経路を発見： 23県通過
さらに長い経路を発見： 24県通過
さらに長い経路を発見： 25県通過
さらに長い経路を発見： 26県通過
さらに長い経路を発見： 27県通過
さらに長い経路を発見： 28県通過
探索終了。1932725939本の経路を調査
条件を満たす経路は見つかりませんでした。
9県を通過する経路が最長経路です
経路：
新潟県 -> 山形県 -> 秋田県 -> 青森県 -> 岩手県 -> 宮城県 -> 福島県 -> 栃木県 -> 埼玉県 -> 長野県 -> 岐阜県 -> 石川県 -> 富山県 -> 福井県 -> 滋賀県 -> 京都府 -> 兵庫県 -> 大阪府 -> 和歌山県 -> 奈良県 -> 三重県 -> 愛知県 -> 静岡県 -> 山梨県 -> 神奈川県 -> 東京都 -> 千葉県 -> 茨城県 -> 群馬県
探索時間:2368.65秒
終了

新潟県-鹿児島県間の最長経路探索

8476 秒（二時間半）

さらに長い経路を発見： 38県通過
さらに長い経路を発見： 39県通過
さらに長い経路を発見： 40県通過
さらに長い経路を発見： 41県通過
さらに長い経路を発見： 42県通過
探索終了。6399944595本の経路を調査
条件を満たす経路は見つかりませんでした。
42県を通過する経路が最長経路です
経路：
新潟県 -> 山形県 -> 秋田県 -> 青森県 -> 岩手県 -> 宮城県 -> 福島県 -> 栃木県 -> 埼玉県 -> 山梨県 -> 静岡県 -> 神奈川県 -> 東京都 -> 千葉県 -> 茨城県 -> 群馬県 -> 長野県 -> 愛知県 -> 三重県 -> 滋賀県 -> 岐阜県 -> 富山県 -> 石川県 -> 福井県 -> 京都府 -> 奈良県 -> 和歌山県 -> 大阪府 -> 兵庫県 -> 徳島県 -> 高知県 -> 愛媛県 -> 香川県 -> 岡山県 -> 広島県 -> 鳥取県 -> 島根県 -> 山口県 -> 福岡県 -> 大分県 -> 宮崎県 -> 熊本県 -> 鹿児島県
探索時間:8476.19秒
終了

目標経路長を超えるまで探索

0.47 秒

新潟県からの最長経路探索プログラム
目的地の都道府県名を入力してください： 福島県

目標とする経路長を入力してください
この値を超えるように探索を繰り返します： 15

新潟県から福島県までの経路の探索を開始

参考：最短経路長は1県
さらに長い経路を発見： 13県通過
さらに長い経路を発見： 19県通過
433633本の経路を調査
条件達成。 19県を通過する経路を発見
経路：
新潟県 -> 富山県 -> 長野県 -> 岐阜県 -> 滋賀県 -> 京都府 -> 奈良県 -> 和歌山県 -> 三重県 -> 愛知県 -> 静岡県 -> 山梨県 -> 神奈川県 -> 東京都 -> 埼玉県 -> 千葉県 -> 茨城県 -> 群馬県 -> 栃木県 -> 福島県
探索時間:0.47秒
終了

最短経路長未満の目標経路長を指定した場合の挙動

（最短より短い経路の探索を求められた場合の例外処理）

新潟県からの最長経路探索プログラム
目的地の都道府県名を入力してください：東京都

目標とする経路長を入力してください
この値を超えるように探索を繰り返します：2

新潟県から東京都までの経路の探索を開始

参考：最短経路長は3県
目標経路長の2県は最短経路長を下回っています。

最短経路：
新潟県 -> 群馬県 -> 埼玉県 -> 東京都
終了

長い。

<計算量>

最長経路探索では全経路を探索する。これは膨大な計算量を必要とすることが分かった。

目標経路長探索では一部の経路のみ探索する。これは全経路探索に比べ計算量が小さいことが分かった。

また、選択する目的地によって経路数が異なり、それに伴い計算時間が決定することも分かった。

<要因>

タスクマネージャを用いて簡単に確認すると、CPU 使用率は低く、メモリ使用率が顕著に大きいことが分かった。

ブラウザなど、他に開いているタスクが計算時間に影響していると分かった。

これらのことから、計算時間はメモリの大きさに依存している可能性があると考えた。

改良

少ない経路本数の目的地を選択しても 9 分、調子が悪いと二時間半かかるプログラムは実用面の観点から非常に品質が悪いと言える。

そのため処理が速くなるように改良したい。

1. マルチスレッド（並列処理）およびマルチプロセス（並行処理）

メモリ使用率が高いことから、並列/並行処理を用いてもメモリ不足で速度が向上しない

と考えた。よって本稿ではこの手法を選択しない。

処理速度を上げるため、主にメモリ効率の向上に重点を置いた。

2. 集合 set で検索

都道府県の隣接関係を表現するグラフは循環する経路を含む。そのため、無限ループする経路を避けて一筆書きの経路のみを探索する処理を必要とする。

探索時に、既に訪問したノードであるか検索する必要があるのだが、これをリストで管理すると効率が悪い。

集合 set は要素に重複が無く、順番を気にしないデータ構造である。リスト型では in 演算の計算量が線形 $O(n)$ であるのに対し、集合型では in 演算が定数オーダー $O(1)$ であることから計算量を削減できると考えた。

しかし、実行時間にはほとんど違いがみられなかった。

```
visited = []      # 訪問済みの場所を記録、無限ループを防ぐ
visited.append([start])
```

↓

```
#listではなくsetを用いることで検索の計算を高速化
visited = set()    # 訪問済みの場所を記録、無限ループを防ぐ
visited.add(start) #listの場合における visited.append([start])に相当
```

3. 都道府県番号で管理

メモリを圧迫している要因は、スタックのデータ構造が保存している大量の経路である。そこで、経路を表現するデータの効率を改善しようと試みた。

それが、数字での管理である。日本語の文字ではなく 1~47 の数字で管理することでメモリ効率が向上すると考えた。

以下にコードの一部を示す。

```

39: {36, 38}, # 高知県
40: {35, 41, 43, 44}, # 福岡県
41: {40, 42}, # 佐賀県
42: {41}, # 長崎県
43: {40, 44, 45, 46}, # 熊本県
44: {40, 43, 45}, # 大分県
45: {43, 44, 46}, # 宮崎県
46: {43, 45, 47}, # 鹿児島県
47: {46} # 沖縄県
}

# 都道府県名と番号の対応関係
self.id_to_name = {
    1: '北海道', 2: '青森県', 3: '岩手県', 4: '宮城県', 5: '秋田県',
    6: '山形県', 7: '福島県', 8: '茨城県', 9: '栃木県', 10: '群馬県',
    11: '埼玉県', 12: '千葉県', 13: '東京都', 14: '神奈川県', 15: '新潟県',
    16: '富山県', 17: '石川県', 18: '福井県', 19: '山梨県', 20: '長野県',
    21: '岐阜県', 22: '静岡県', 23: '愛知県', 24: '三重県', 25: '滋賀県',
    26: '京都府', 27: '大阪府', 28: '兵庫県', 29: '奈良県', 30: '和歌山県',
    31: '鳥取県', 32: '島根県', 33: '岡山県', 34: '広島県', 35: '山口県',
    36: '徳島県', 37: '香川県', 38: '愛媛県', 39: '高知県', 40: '福岡県',
    41: '佐賀県', 42: '長崎県', 43: '熊本県', 44: '大分県', 45: '宮崎県',
    46: '鹿児島県', 47: '沖縄県'
}

# 名前から番号への逆引き辞書
self.name_to_id = {v: k for k, v in self.id_to_name.items()}

隣接する都道府県のIDを取得する関数
def get_neighbors(self, pref_id):

```

これにより内部では数字で経路を管理し、入出力のみ都道府県名を使用するようにした。
しかし、実行時間に大きな違いはみられなかった。

4. 言語の変更

一般に、高級な言語ほど人間にとっては扱いやすく、低級な言語ほど機会にとって扱いやすい傾向にあるはずだ。

(もちろんここでのいう言語とはプログラム言語であり、自然言語に優劣を唱えるものではないことをご承知いただきたい。)

そこで、python にて実装したプログラムを C 言語で実装してみた。

処理能力が向上すると予想する。

またここでは、都道府県番号で経路を管理する方式を採用している。

新潟県-福島県間の最長経路探索

78 秒：処理速度 7 倍

新潟県からの最長経路探索プログラム
目的地の都道府県名を入力してください：福島県

目標とする経路長を入力してください
この値を超えるように探索を繰り返します：47

新潟県から福島県までの経路の探索を開始

参考：最短経路長は1県
さらに長い経路を発見：13県通過 (111475本探索)
さらに長い経路を発見：15県通過 (748562本探索)
さらに長い経路を発見：19県通過 (808708本探索)
さらに長い経路を発見：20県通過 (809246本探索)
さらに長い経路を発見：21県通過 (817121本探索)
さらに長い経路を発見：22県通過 (853586本探索)
さらに長い経路を発見：23県通過 (956006本探索)

探索終了。全692294242本の経路を調査しました。
見つかった最長経路は 23県通過です。

経路：

新潟県 -> 富山県 -> 長野県 -> 岐阜県 -> 石川県 -> 福井県 -> 滋賀県 -> 京都府 -> 兵庫県 -> 大阪府 -> 和歌山県 -> 奈良県 -> 三重県 -> 愛知県 -> 静岡県 -> 山梨県 -> 神奈川
県 -> 東京都 -> 埼玉県 -> 千葉県 -> 茨城県 -> 群馬県 -> 栃木県 -> 福島県

探索時間:77.72秒

終了

新潟県-鹿児島県間の最長経路探索

507 秒：処理速度 17 倍

新潟県からの最長経路探索プログラム
目的地の都道府県名を入力してください：鹿児島県

目標とする経路長を入力してください
この値を超えるように探索を繰り返します：47

新潟県から鹿児島県までの経路の探索を開始

参考：最短経路長は10県
さらに長い経路を発見：17県通過 (18本探索)
さらに長い経路を発見：18県通過 (20本探索)
さらに長い経路を発見：20県通過 (47本探索)
さらに長い経路を発見：21県通過 (49本探索)
さらに長い経路を発見：22県通過 (171本探索)
さらに長い経路を発見：23県通過 (1787本探索)
さらに長い経路を発見：24県通過 (27561本探索)
さらに長い経路を発見：25県通過 (33842本探索)
さらに長い経路を発見：26県通過 (33964本探索)
さらに長い経路を発見：27県通過 (40363本探索)
さらに長い経路を発見：28県通過 (292741本探索)
さらに長い経路を発見：29県通過 (1237640本探索)
さらに長い経路を発見：30県通過 (1482608本探索)
さらに長い経路を発見：31県通過 (1727576本探索)
さらに長い経路を発見：32県通過 (1972544本探索)
さらに長い経路を発見：33県通過 (2216061本探索)
さらに長い経路を発見：34県通過 (2453059本探索)
さらに長い経路を発見：35県通過 (2453181本探索)
さらに長い経路を発見：36県通過 (2459580本探索)
さらに長い経路を発見：37県通過 (2702985本探索)
さらに長い経路を発見：38県通過 (1452514289本探索)
さらに長い経路を発見：39県通過 (2200302009本探索)
さらに長い経路を発見：40県通過 (2948089729本探索)
さらに長い経路を発見：41県通過 (3695877438本探索)
さらに長い経路を発見：42県通過 (4443665148本探索)

探索終了。全5188435132本の経路を調査しました。
見つかった最長経路は 42県通過です。

経路：

新潟県 -> 山形県 -> 秋田県 -> 青森県 -> 岩手県 -> 宮城県 -> 福島県 -> 栃木県 -> 埼玉県 -> 山梨県 -> 静岡県 -> 神奈川県 -> 東京都 -> 千葉県 -> 茨城県 -> 群馬県 -> 長野県
-> 愛知県 -> 三重県 -> 滋賀県 -> 岐阜県 -> 富山県 -> 石川県 -> 福井県 -> 京都府 -> 奈良県 -> 和歌山県 -> 大阪府 -> 兵庫県 -> 徳島県 -> 高知県 -> 愛媛県 -> 香川県 -> 岡山
県 -> 広島県 -> 鳥取県 -> 島根県 -> 山口県 -> 福岡県 -> 大分県 -> 宮崎県 -> 熊本県 -> 鹿児島県

探索時間:507.06秒

終了

このように処理速度が向上することが分かった。

5. <ヒューリスティック探索> 反復局所探索法

最長経路探索プログラムを何度も実行して気付いたことがある。

真の最長経路が発見された後、それが最長であることを証明するために残りの全経路を
探索する処理に時間を要しているということだ。

そこで、この経験則をプログラムに実装したい。

このような工夫をヒューリスティックと呼ぶらしい。

ここでいうヒューリスティックとは、経験則を導入して局所解を許容する手続きのことである。アルゴリズムとは対比して扱われることも多い。

以下の手続きを実装した。

=====

1, 暫定最長経路を発見

2, それまでの探索回数を記録（=>この回数を A とする）

3, 探索回数が A の n 倍に達してもより良い経路を発見できなかった場合、局所解を最長経路として採用する

=====

この処理は反復局所探索法というヒューリスティック探索に該当するものと思われる。

全目的地に対して適切な定数 n を取ることで最適解と同じ局所解を常に得ることができるはずだ。とりあえず定数 n には 10 を代入してみる。

<実行結果と考察>

成功した例：処理速度 20 倍

```
新潟県からの最長経路探索プログラム
目的地の都道府県名を入力してください：福島県

目標とする経路長を入力してください
この値を超えるように探索を繰り返します：47

新潟県から福島県までの経路の探索を開始

参考：最短経路長は1県
さらに長い経路を発見：13県通過（129664本探索）
さらに長い経路を発見：19県通過（889353本探索）
さらに長い経路を発見：20県通過（889890本探索）
さらに長い経路を発見：21県通過（897763本探索）
さらに長い経路を発見：22県通過（963738本探索）
さらに長い経路を発見：23県通過（1244153本探索）

暫定最長経路を発見後に10倍の本数の経路を探索しましたが、より良い経路は見つかりませんでした。
探索を打ち切り、現在の最長経路を局所解とします。
見つかった最長経路は 23県通過です。

経路：
新潟県 -> 富山県 -> 長野県 -> 岐阜県 -> 石川県 -> 福井県 -> 滋賀県 -> 京都府 -> 兵庫県 -> 大阪府 -> 和歌山県 -> 奈良県 -> 三重県 -> 愛知県
-> 静岡県 -> 山梨県 -> 神奈川県 -> 東京都 -> 埼玉県 -> 千葉県 -> 茨城県 -> 群馬県 -> 栃木県 -> 福島県
探索時間:25.30秒
終了
```

失敗した例：局所解と最適解が大きく異なる

```
新潟県からの最長経路探索プログラム
目的地の都道府県名を入力してください：鹿児島県

目標とする経路長を入力してください
この値を超えるように探索を繰り返します： 47

新潟県から鹿児島県までの経路の探索を開始

参考：最短経路長は10県
さらに長い経路を発見： 17県通過 (18本探索)
さらに長い経路を発見： 18県通過 (20本探索)
さらに長い経路を発見： 20県通過 (47本探索)
さらに長い経路を発見： 21県通過 (49本探索)
さらに長い経路を発見： 22県通過 (171本探索)
さらに長い経路を発見： 23県通過 (1789本探索)

暫定最長経路を発見後に10倍の本数の経路を探索しましたが、より良い経路は見つかりませんでした。
探索を打ち切り、現在の最長経路を局所解とします。
見つかった最長経路は 23県通過です。

経路：
新潟県 -> 富山県 -> 長野県 -> 愛知県 -> 三重県 -> 和歌山県 -> 奈良県 -> 大阪府 -> 京都府 -> 兵庫県 -> 徳島県 -> 高知県 -> 愛媛県 -> 香川県
-> 岡山県 -> 広島県 -> 鳥取県 -> 島根県 -> 山口県 -> 福岡県 -> 大分県 -> 宮崎県 -> 熊本県 -> 鹿児島県
探索時間:0.05秒
終了
```

新潟県-福島県間は真の最長経路長 23 と同じ解が得られている。

しかし新潟県-鹿児島県間では最長経路長とは大きく異なる解が得られてしまった。

この違いには、スタックのデータ構造が影響していると考える。スタックのデータ構造ではリストの後ろから順に参照して探索を進める。つまり、都道府県が大きい方から順に探索するのである。

ここで、鹿児島県までの真の最長経路は{新潟県->山形県->秋田県->青森県……->鹿児島県}と、一旦北上してから鹿児島県へ向かっている。つまり、都道府県番号の小さい経路が最長経路であり、発見が遅れる。

このことから、はじめに北上してから南へ向かう経路が最長経路となる目的地には対して、このヒューリスティックは効果を発揮できないと考える。

この探索法を全都道府県で利用するための処理を考察すると、

出発地と目的地の都道府県番号の関係を比較して、探索する経路の順番を[昇順][降順]と変更する処理が有効ではないかと考える。

6. <ヒューリスティック探索>焼きなまし法

焼きなまし法、あるいはアニーリングと呼ばれるヒューリスティック探索を実装してみた。

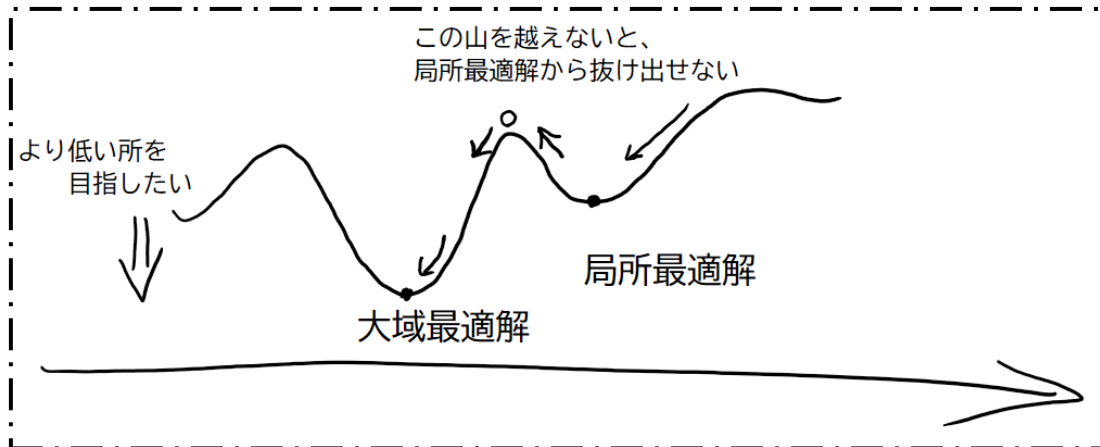
焼きなまし法とは、確率的な最適化手法で、「温度」と「冷却率」というパラメータを導入して最適解を探索する。(金属の焼きなまし過程が由来となっている。)

温度は安定性を表しており、温度が高いほど悪い解へ遷移する確率が高くなる。

温度が低いほど、より良い解へのみ遷移するようになる。

ランダムに初期解を決定し、探索を進めるごとに徐々に温度を小さくしていく。

すると、はじめは大胆に遷移し局所最適解の谷を抜け出す確率が高い。よって広く探索できる。次第に温度が低くなるにつれて局所最適解の周辺に落ち着き微調整するようになる。厳密な解にたどり着くとは限らないが、より良い局所最適解を探索することができるようになる。



(図 2.1 次元の解空間における、最適解のイメージ図)

焼きなまし法を用いた最長経路探索プログラムの実行結果を以下に示す。

新潟県からの最長経路探索プログラム (焼きなまし法版)

目的地の都道府県名を入力してください: 福島県

新潟県から福島県までの経路の探索を開始します。

参考: 最短経路は 1 県を通過します。

-> 新潟県 -> 福島県

焼きなまし法による最長経路探索を開始...

パラメータ: 初期温度=100.0, 冷却率=0.9995, 試行回数=300000

初期経路を発見: 2県通過

探索が終了しました。

発見された最長経路 (2 県通過):

新潟県 -> 山形県 -> 福島県

探索時間: 0.21 秒

```
新潟県からの最長経路探索プログラム（焼きなまし法版）
=====
目的地の都道府県名を入力してください：福島県

新潟県から福島県までの経路の探索を開始します。

参考：最短経路は 1 県を通過します。
-> 新潟県 -> 福島県
-----
焼きなまし法による最長経路探索を開始...
パラメータ：初期温度=10.0, 冷却率=0.9995, 試行回数=300000
初期経路を発見：3県通過
試行 1: さらに長い経路を発見：12県通過
試行 26: さらに長い経路を発見：16県通過
試行 28: さらに長い経路を発見：18県通過
試行 46: さらに長い経路を発見：22県通過
試行 52: さらに長い経路を発見：23県通過
-----
探索が終了しました。

発見された最長経路（23 県通過）:
新潟県 -> 長野県 -> 富山県 -> 石川県 -> 福井県 -> 京都府 -> 兵庫県 -> 大阪府 -> 奈良県 -> 和歌山県 -> 三重県 -> 滋賀県 -> 岐阜県 -> 愛知県 ->
> 静岡県 -> 山梨県 -> 神奈川県 -> 東京都 -> 千葉県 -> 埼玉県 -> 栃木県 -> 群馬県 -> 茨城県 -> 福島県

探索時間：0.21 秒
```

```
新潟県からの最長経路探索プログラム（焼きなまし法版）
=====
目的地の都道府県名を入力してください：鹿児島県

新潟県から鹿児島県までの経路の探索を開始します。

参考：最短経路は 10 県を通過します。
-> 新潟県 -> 富山県 -> 福井県 -> 京都府 -> 兵庫県 -> 鳥取県 -> 島根県 -> 山口県 -> 福岡県 -> 熊本県 -> 鹿児島県
-----
焼きなまし法による最長経路探索を開始...
パラメータ：初期温度=100.0, 冷却率=0.9995, 試行回数=300000
初期経路を発見：21県通過
試行 67: さらに長い経路を発見：23県通過
試行 68: さらに長い経路を発見：25県通過
試行 342: さらに長い経路を発見：26県通過
試行 815: さらに長い経路を発見：28県通過
試行 849: さらに長い経路を発見：30県通過
試行 1527: さらに長い経路を発見：31県通過
試行 1538: さらに長い経路を発見：32県通過
試行 3623: さらに長い経路を発見：33県通過
試行 4359: さらに長い経路を発見：34県通過
試行 4854: さらに長い経路を発見：35県通過
試行 5140: さらに長い経路を発見：36県通過
試行 73192: さらに長い経路を発見：37県通過
試行 133156: さらに長い経路を発見：38県通過
試行 134501: さらに長い経路を発見：39県通過
試行 161175: さらに長い経路を発見：40県通過
-----
探索が終了しました。

発見された最長経路（40 県通過）:
新潟県 -> 山形県 -> 秋田県 -> 岩手県 -> 宮城県 -> 福島県 -> 茨城県 -> 群馬県 -> 長野県 -> 埼玉県 -> 千葉県 -> 東京都 -> 山梨県 -> 神奈川県 ->
> 静岡県 -> 愛知県 -> 岐阜県 -> 石川県 -> 富山県 -> 福井県 -> 滋賀県 -> 三重県 -> 京都府 -> 奈良県 -> 和歌山県 -> 大阪府 -> 兵庫県 -> 岡山県 ->
> 香川県 -> 徳島県 -> 高知県 -> 愛媛県 -> 広島県 -> 鳥取県 -> 島根県 -> 山口県 -> 福岡県 -> 大分県 -> 宮崎県 -> 熊本県 -> 鹿児島県

探索時間：5.50 秒
```

アニーリングでは乱数を用いて初期解や遷移確率を処理しているため、結果にばらつきが生じる。また、パラメータにも影響される。

そのためテストコードを作成し、実行結果をまとめた。

<テストコードの実行結果>

全テストシナリオ完了：結果サマリー

出発地：新潟県、各シナリオの試行回数：500回、1試行あたりの計算回数：30000回

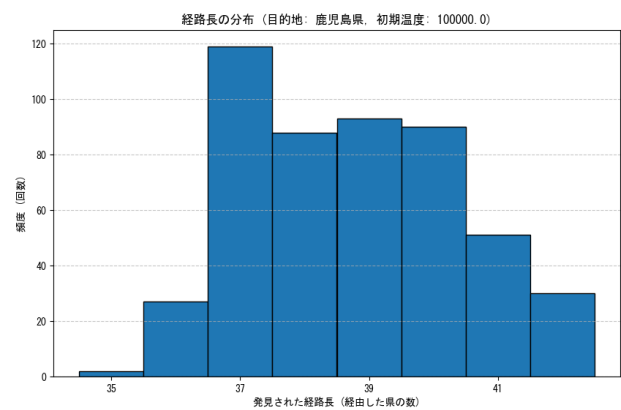
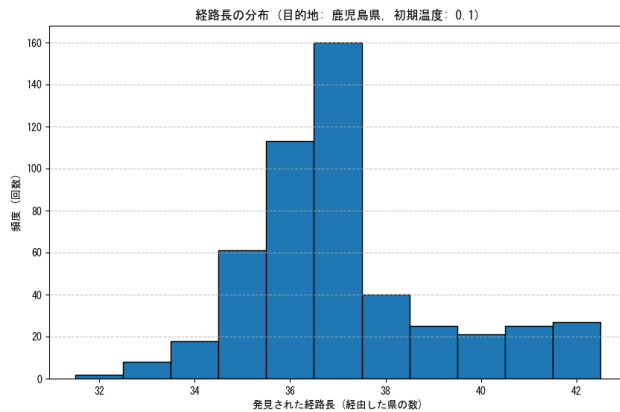
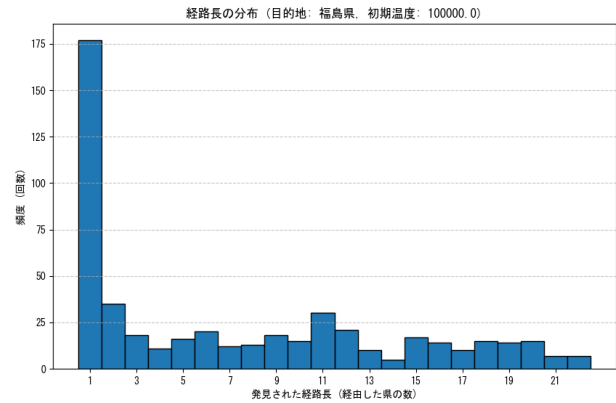
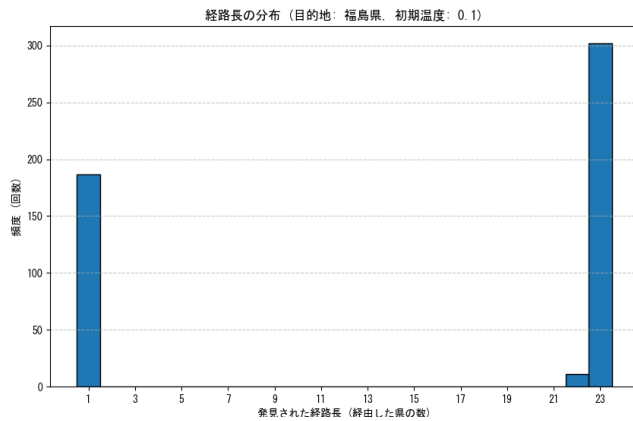
目的地：福島県 | 初期温度：0.1 -> 平均長：14.75, 分散：112.98, 最大長：23 (平均時間/試行：0.3963秒)
 目的地：福島県 | 初期温度：100000.0 -> 平均長：7.23, 分散：44.11, 最大長：22 (平均時間/試行：0.0211秒)
 目的地：鹿児島県 | 初期温度：0.1 -> 平均長：37.11, 分散：4.19, 最大長：42 (平均時間/試行：0.5797秒)
 目的地：鹿児島県 | 初期温度：100000.0 -> 平均長：38.73, 分散：2.74, 最大長：42 (平均時間/試行：0.6084秒)

分布グラフを生成中...

-> グラフを保存しました：distribution_福島県_temp_0.1.png
 -> グラフを保存しました：distribution_福島県_temp_100000.0.png
 -> グラフを保存しました：distribution_鹿児島県_temp_0.1.png
 -> グラフを保存しました：distribution_鹿児島県_temp_100000.0.png

グラフの生成が完了しました。

テスト全体の総実行時間：804.51秒



	初期温度 0.1	初期温度 100000
福島県	○厳密解が得られやすい	△最短経路や局所最適に陥りやすい
鹿児島県	△37 周辺に分布している	○37 以上に集中している(悪い解が少ない)

近い目的地では、初期温度が小さい方が全体最適解を得られやすかった。
また、初期温度を変えても最悪な解の出現回数が同程度であった。
遠い目的地では、初期温度が大きい方が最適解に近い局所解を得られやすかった。
また、初期温度を変えても全体最適解の出現回数が同程度であった。

目的地が近く初期温度が高いと、厳密解に到達した後も不安定な状態が続き、厳密解の谷を抜け出してしまうのではないかと考える。

目的地が遠すぎると、経路数が多く複雑で局所最適が多い。初期温度が低いと局所最適に陥りやすく、初期温度を高くすることで局所最適から抜け出すことができると考える。

また、焼きなまし法の欠点を感じた。

目的地が近すぎると、探索の初期経路に最短経路を採用してしまい効率が悪くなる。特に、新潟県-福島県間のような隣接ノードで最短経路が選ばれると、直接接続されて途中経路の変更ができない。そのため焼きなまし法が正常に動作しない。

焼きなまし法ではこのように、探索する二点の位置関係で性能や適切なパラメータが異なるとわかった。

これらのことから、最短経路長に応じて適切な初期温度を自動的に選択する関数を実装することですべての都道府県に対して厳密解に近い局所解を得ることができると考える。

7. 量子アニーリングと QUBO 行列の考察

量子アニーリングの利用可能性について、この項では考察する。

量子アニーリングとは、組合せ最適化問題を解く手法である。量子の性質を利用し、多数の選択肢から最小のコストとなる選択肢を探索するものである。決して疑似科学ではない点だけをご理解いただきたい。

私は 2024 年度の年刊 EDP にて、量子コンピュータおよび量子アニーリングについてまとめ、実践にも挑戦した。

三日坊主は量子計算の夢を見られるのか

遠雷亭

本稿は量子コンピュータの理解と実践に三日間取り組んだ私の体験記です。

目次

- 1 はじめに
- 2 ゆる〜く概要説明
- 3 実践編 量子コンピュータ触ってみた
- 4 量子コンピュータ体験ゲーム
- 5 数々の困難
- 6 おわりに

ひとまずこの記事をおさらいする。

<概要>

一般に量子コンピュータと呼ばれるものには、量子ゲート方式と量子アニーリング方式の二方式があり、動作原理や使い方が異なる。

本稿では量子アニーリング方式にのみ言及する。

量子アニーリングは、組み合わせ最適化問題に特化した実験装置だ。前述したヒューリスティックの焼きなまし法と似ている。

ざっくりと計算過程を以下に示す。

- ・初期状態では量子ビットを不安定な状態にさせる。
- ・そこから徐々に量子ビットを安定させる。
- ・最終的な量子ビットの状態が局所解を表現している。

これは「量子ビットが安定状態へ向かう時に全体のエネルギーが最小になる状態へ向かう物理現象」を、「コスト関数の解空間において低い値へ向かうこと」に対応させており、イメージ図はヒューリスティックのアニーリング法と同様になる。

<実践編>

実践というのは、クラウドサービスの利用である。

高価で場所をとるため量子コンピュータをお持ちでないご家庭も多いかもしれないが、量子コンピュータや量子アニーリングはクラウドサービスがいくつか提供されており、時間制限付きで無料利用できるものも多い。

どこにしようと誰だろうと、ネット環境さえあれば量子アニーリングは利用できるということだ。

しかし、この量子アニーリングは QUBO 行列という行列を入力値として用意しなければならない。古典コンピュータのように言語が充実しているものではなく、どのような問題も QUBO 行列に変換して入力する必要がある。

ここまでが” 三日坊主は量子計算の夢を見られるのか” の振り返りである。

<QUBO 行列>

まず QUBO 行列とは、0 と 1 の二種類の値のみを取る変数でコスト関数が決定するとき、コスト関数を最小にする組み合わせの変数の組を表現するための行列である。

まずは本稿で取り扱っている問題をコスト関数 H で表現する。

コスト関数は、目的に沿った入力であるほど値が小さくなる関数である。

問題を解くうえでのルールや取ってはいけない解を避けるようにするために、制約条件を導入する。制約条件に該当する入力に対して、非常に大きな値の出力が返されるような式を加える。

まず目的関数から見ていこう。コスト関数では、小さい出力ほど求めたい解になるようにすべきだ。47 都道府県の重み無しグラフにおける最長経路探索問題においては、経路長が長いほど求めたい解であるといえる。よって、経路の長さ $\times (-1)$ の値を返す式が目的関数として使える。この目的関数を実装していく。

まず都道府県の隣接関係を表現した 47×47 の正方行列を用意する。

行の u 番目、列の v 番目をそれぞれ都道府県番号と対応させ、隣接していれば 1、そうでなければ 0 とする。(例えば、青森県は北海道、岩手県、秋田県とだけ隣接しているから、二行目は $[1, 0, 1, 0, 1, 0, 0, \dots, (\text{省略}), 0]$ となる。このような行が 47 行連なった行列である。)

この行列を $A_{u,v}$ とおく。

都道府県番号(0~46)を v として、経路上で何番目に通るのかという順序を i (0~47)とした変数を $x_{v,i}$ とすると、

$$H_{\text{obj}} = - \sum_{i=0}^{N-2} \sum_{u=0}^{N-1} \sum_{v=0}^{N-1} A_{uv} x_{u,i} x_{v,i+1}$$

と表現できる。経路が長ければ長いほど、この関数は小さい値をとる。

ここで一旦、都道府県の隣接関係を表現する行列 $A_{u,v}$ をつくるプログラムを実装する。

```

#都道府県の隣接関係を 47*47 正方行列で出力するプログラム
import numpy as np
np.set_printoptions(threshold=np.inf)
# 都道府県のリスト（行列のインデックス 0~46 に対応）
prefectures_map = ['北海道', '青森県', '岩手県', '宮城県', '秋田県', '山形県',
                   (省略)'新潟県', (省略), '沖縄県']

# 都道府県名とインデックスを紐付ける辞書
pref_to_idx = {name: i for i, name in enumerate(prefectures_map)}

# 隣接データ
adjacency_data = { '北海道': ['青森県'],
                   '青森県': ['岩手県', '秋田県', '北海道'],
                   '岩手県': ['青森県', '宮城県', '秋田県'],
                   '宮城県': ['岩手県', '秋田県', '山形県', '福島県'],
                   '秋田県': ['青森県', '岩手県', '宮城県', '山形県'],
                   '山形県': ['秋田県', '宮城県', '福島県', '新潟県'],
                   '福島県': ['宮城県', '山形県', '新潟県', '茨城県', '栃木県'],
                   (省略)
                   '新潟県': ['山形県', '福島県', '群馬県', '長野県', '富山県'],
                   (省略)
                   '沖縄県': ['鹿児島県']
                   }

# 47x47 のゼロ行列を初期化
adj_matrix = np.zeros((47, 47), dtype=int)

# 隣接リストから行列を生成
for pref, neighbors in adjacency_data.items():
    if pref not in pref_to_idx: continue
    pref_idx = pref_to_idx[pref]
    for neighbor in neighbors:
        if neighbor not in pref_to_idx: continue
        neighbor_idx = pref_to_idx[neighbor]
        adj_matrix[pref_idx, neighbor_idx] = 1
print(adj_matrix)

```

[illegible]

このようになる。しかしこれでは、制約条件を設定できていない。ここから、いくつかの制約条件を課す。

条件 1:連続な経路のみ選択

当たり前のことだが、経路は連続していなければならない。ワープしてはいけない。
このことを制約条件として加える。

$$H_{c1} = P_1 \sum_{i=1}^{N-1} \left(\sum_{u=0}^{N-1} \sum_{v=0}^{N-1} A_{uv} x_{u,i-1} x_{v,i} - 1 \right)^2$$

ここでPはペナルティパラメータであり、十分に大きな正の数。
Nは要素数47。また、前述のとおりu,vは都道府県の要素番号でiは経路の順序である。
i-1番目の経路に何かが存在するとき、i番目の経路に何かがあってもこの項は0をとる。
つまり目的関数の邪魔をしない。
しかし一つ前の経路に何もなかったとき、この項は正の数×Pで大きな値を取る。
この項によって経路が連続しない組み合わせを回避することができる。

条件 2:一度に訪れる地点は一つのみ

これも当たり前のことだが、一度に移動できる地点は一つだけである。分身はできない。
このことも制約条件として加える。

$$H_{c2} = P_2 \sum_{i=0}^{N-1} \left(\sum_{v=0}^{N-1} x_{v,i} - 1 \right)^2$$

これによりi番目の経路で複数のノードへ進もうとする選択枝を邪魔することができる。

条件 3:一筆書きの経路のみ選択

各地点は一度しか経由しないようにしなければならない。これも制約条件として加える。

案 1

$$H_{c3} = P_3 \sum_{v=0}^{N-1} \left(\sum_{i=0}^{N-1} x_{v,i} - 1 \right)^2$$

H_{c2} と似ているが、この項は、全経路を通して各ノードの登場回数が1であることを強制する機能を持つ。つまり全ノードを通過しようとしている。そのような経路は存在しないため、一筆書きを強制しつつ、全てのノードを訪問しなくても許すソフトな制約条件の項とする。

そのため、ペナルティパラメータ P_3 は他よりも小さい適切な値を設定する必要がある。
他に良い制約条件の式がないか検討を重ねる。

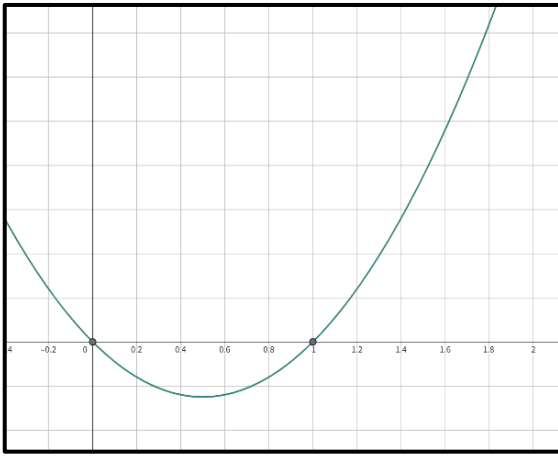
案 2

以下の項を設定すれば各ノードの登場回数が1か0であればよいことになる。
しかし、max関数は離散的な関数であり QUBO 行列や量子アニーリングに適用できない。

$$H'_{c3} = P'_3 \sum_{v=0}^{N-1} \max\left(0, \sum_{i=0}^{N-1} x_{v,i} - 1\right)^2$$

案 3

問題を整理すると、あるノードへの訪問回数が0と1のとき->0を返し、それ以上のとき->正の値を返すという関数が求められている。
0と1の総和で表現しているため離散的だが、もし連続であれば以下のに示すような関数が適切である。



$$\begin{aligned} y &= x^2 - x \\ &= x(x - 1) \end{aligned}$$

これをコスト関数で表現すると、

$$H''_{c3} = P''_{c3} \sum_{v=0}^{N-1} \left(\sum_{i=0}^{N-1} x_{v,i} \right) \left(\sum_{i=0}^{N-1} x_{v,i} - 1 \right)$$

となる。これは、訪問回数が0のとき1のときは目的関数を阻害せず、訪問回数が2より大きい組み合わせを制限する関数として機能する。

複数回の訪問を禁止し、一筆書きを強制する制約条件ができる。

よってこの案を採択し、 H_{c3} は以下の式で表現する。

$$H_{c3} = P_{c3} \sum_{v=0}^{N-1} \left(\sum_{i=0}^{N-1} x_{v,i} \right) \left(\sum_{i=0}^{N-1} x_{v,i} - 1 \right)$$

条件 4：始点を新潟県とする

これまでの三つの条件下では、47都道府県のどこか2地点を端点に選んだ場合の一筆書き最長経路の探索であった。“全点对最長経路探索”といえる。

4つ目に、新潟県を始点とするための制約条件を加える。

$$H_{c4} = P_4 (x_{s,0} - 1)^2$$

このとき s は新潟県の要素番号であるとする。

0 番目の経路が s 、つまり始点が新潟県でなければ大きな値で邪魔する機能を持つ項である。

ここまでの項の総和は、

$$H = H_{obj} + H_{c1} + H_{c2} + H_{c3} + H_{c4}$$

となる。ここで各項を見直してみると、

H_{c1} は $(\sum_{u=0}^{N-1} x_{u,i-1} - 1)^2$ の因数を含み

H_{c2} は $(\sum_{v=0}^{N-1} x_{v,i} - 1)^2$ の因数を含み単項式である。

このように似た因数を持っており、違いは $i=0$ のみである。

H_{c1} は $i=0$ のときのみ制限していない。

逆に H_{c2} は $i=0$ でのみ意味のある項である。

そして H_{c4} は始点のノードを新潟県のように制限する項である。

よって H_{c1} と H_{c4} の制限と被っていることから、 H_{c2} の項は冗長項となり不要である。

よってコスト関数は以下の式になる。

$$\begin{aligned} H = & - \sum_{i=0}^{N-2} \sum_{u=0}^{N-1} \sum_{v=0}^{N-1} A_{uv} x_{u,i} x_{v,i+1} \\ & + P_1 \sum_{i=1}^{N-1} \left(\sum_{u=0}^{N-1} \sum_{v=0}^{N-1} A_{uv} x_{u,i-1} x_{v,i} - 1 \right)^2 \\ & + P_{c3} \sum_{v=0}^{N-1} \left(\sum_{i=0}^{N-1} x_{v,i} \right) \left(\sum_{i=0}^{N-1} x_{v,i} - 1 \right) \\ & + P_4 (x_{s,0} - 1)^2 \end{aligned}$$

変数を再度示すと、

N は要素数 47

u, v は要素番号 0~46

i は経路の順序

s は始点にしたい要素の要素番号

A_{uv} は隣接関係を 0,1 で表現した 47*47 の正方行列

$P_{1,3,4}$ 大きな正の数 ($0 < P_3 < P_1 = P_4$)

これが、新潟県から 47 都道府県への最長経路探索のコスト関数である。

また、これは終点を設定していない“単一起点最長経路探索”である。他のプログラムのように、目的地の入力を受け付けることはできない。

このコスト関数をとあるルールに則り整理することで $(47)^2 \times (47)^2$ の QUBO 行列へ変換することができる。

その後、D-wave が提供する量子アニーリングマシンのクラウドサービス “Leap” に接続し、QUBO 行列を入力する。

QUBO 行列に対する出力を解析することで新潟県からどこかへの最長経路探索を実行できる。

しかし本稿では、コスト関数の定式化までにとどまることとする。
コスト関数から QUBO 行列への変換と QUBO 行列の実行、出力の解析は扱わない。
本稿の場合、上記の手順を踏み出力を得た後、解析結果の検証まで行う必要がある。
(具体的に検証というのは、厳密解であるかの確認や処理時間の比較、適切な定数項の検討などが想定される。)
これらには多くの工数を必要とし、本稿の主眼から大きく乖離すると判断したためである。
これらを今後の課題とする。

三日坊主は量子計算の夢を見られるのか	遠雷亭
本稿は量子コンピュータの理解と実践に三日間取り組んだ私の体験記です。	
目次	
1 はじめに	
2 ゆる〜く概要説明	
3 実践編 量子コンピュータ触ってみた	
4 量子コンピュータ体験ゲーム	
5 数々の困難	
6 おわりに	

⇔2024 年度年刊 EDP に収録

量子アニーリングの使い方が気になる方はこちらをご参照いただきたい。



新潟大学コンピュータクラブ | 制作物一覧

<https://edp.mods.jp/work.html>

8. ライブラリの利用

最後に、グラフ構造を扱うライブラリを探してみた。

ライブラリを利用すればプログラムの作成効率も性能も良くなると予想していた。

関数の中身がブラックボックスになると自分で考えなくなってしまうし、すぐに問題が解決してしまっはせっかくのやる気がもったいない。

自分でいろいろ工夫する過程で学びを得たいという理由でライブラリの利用を最終手段としていた。

Graphillion というライブラリを利用した。

Graphillion は、巨大な数のグラフ集合を効率的に扱うためのライブラリだ。

このライブラリの威力は、ZDD を構築することにある。

<ZDD とは>

ZDD とは、ゼロ抑制決定グラフ (Zero-suppressed Decision Diagram) のことである。

膨大な組み合わせを圧縮、剪定して表現するデータ構造だ。

まず、部分集合を構築する。都道府県は 47 個の要素があるため、2 の 47 乗通りの部分集合で表現される。

ノード (都道府県) ではなくエッジ (経路) に注目し、各エッジを経由するか否かで二分木を構築する。

この二分木のうち不要なものを除去、同じ構造となるものを統合することで二地点を結ぶ経路のみを表現できる。逆を言えば、この操作により目的地へ着かない不要な経路とループする閉路を除くことができる。

<関数の説明>

`GraphSet.set_universe(universe)`

集合を定義

`GraphSet.paths(start, goal)`

第一引数のノードと第二引数のノードを結ぶ ZDD による集合を返す

これにより探索する経路が単純かつ少数になる

`GraphSet.max_iter(weights)`

重みの合計値が最大の経路を返す

ここでは重み無しグラフを用いるため、経路長が最大の経路を返す

以下に実行結果を示す。

Graphillion を用いた都道府県最長経路探索

目的地の都道府県名を入力してください: 福島県

新潟県 から 福島県 までの最長経路を探索します…
23 県を通過する最長経路を発見

最長経路:

新潟県 -> 富山県 -> 石川県 -> 福井県 -> 滋賀県 -> 京都府 -> 兵庫県 -> 大阪府
-> 和歌山県 -> 奈良県 -> 三重県 -> 愛知県 -> 岐阜県 -> 長野県 -> 群馬県 ->
栃木県 -> 埼玉県 -> 山梨県 -> 静岡県 -> 神奈川県 -> 東京都 -> 千葉県 -> 茨城
県 -> 福島県

探索時間: 0.01 秒

Graphillion を用いた都道府県最長経路探索

目的地の都道府県名を入力してください: 鹿児島県

新潟県 から 鹿児島県 までの最長経路を探索します…
42 県を通過する最長経路を発見

最長経路:

新潟県 -> 山形県 -> 秋田県 -> 青森県 -> 岩手県 -> 宮城県 -> 福島県 -> 茨城県
-> 千葉県 -> 東京都 -> 神奈川県 -> 静岡県 -> 山梨県 -> 埼玉県 -> 栃木県 ->
群馬県 -> 長野県 -> 富山県 -> 石川県 -> 福井県 -> 京都府 -> 滋賀県 -> 岐阜県
-> 愛知県 -> 三重県 -> 奈良県 -> 和歌山県 -> 大阪府 -> 兵庫県 -> 徳島県 ->
高知県 -> 愛媛県 -> 香川県 -> 岡山県 -> 広島県 -> 鳥取県 -> 島根県 -> 山口県
-> 福岡県 -> 熊本県 -> 大分県 -> 宮崎県 -> 鹿児島県

探索時間: 0.01 秒

一瞬である。

ZDD で圧縮管理している恩恵で 47 都道府県のグラフ集合でも高速に動作する。

どの目的地に対しても探索時間は 0.01 を返すため、おそらくこの値より小さい実行時間
で探索を完了していると思われる。

今までの苦勞が全て無に帰す性能であった。

(もっとも今回は学習目的であったため無駄な苦勞ではないが。)

結論と展望

本稿で作成したプログラムをまとめる

- 土台となる最も遅いプログラム
- 都道府県番号で管理するプログラム
=> あまり性能が変化しなかった。
- C 言語のプログラム
=> 10 倍程度性能が向上した。
- 反復局所探索法のプログラム
- 焼きなまし法のプログラム
=> ヒューリスティックは適切なパラメータ、適切な条件でのみ厳密解に近づくことがわかった。
- 量子アニーリングを利用するためのプログラム
=> コスト関数の定式化まで行った。
実装まで至らなかった。
- Graphillion ライブラリを利用したプログラム
=> グラフの圧縮と剪定により効率化するライブラリを用いた。
全ての目的地に対して現実的な処理時間で最長経路を求めることができた。

次に、今後の展望をまとめる。

- マルチスレッドおよびマルチプロセス
今回は並列処理の実装は検討のみにとどまった。並列処理が有効である問題とそうでない問題の違いや、並列処理と並行処理の違いなど、最長経路長探索における適切な処理について今後調査したい。
- QUBO 行列の実行
今回は QUBO 行列を実行するまでに至らなかった。
量子アニーリング、シュミレーテッドアニーリングにて実行し、結果をまとめたい。
- 重み付け

本稿で作成したグラフは重み無しグラフであった。

距離や実際の交通状況を反映させた重みづけグラフを利用して経路探索を行うことでより実用的になると考える。

- 別のグラフ

本稿で作成したグラフは 47 都道府県であった。より細かく市区町村でグラフをつくったり、一つの県に絞ったり、地球規模に広げたりなど、あらゆるグラフ集合に対して最長経路探索ができるようにしたい。

本稿の検証で最も性能のよかった” Graphillion ライブラリを利用したプログラム”を用いて最長経路探索の結果をまとめる。

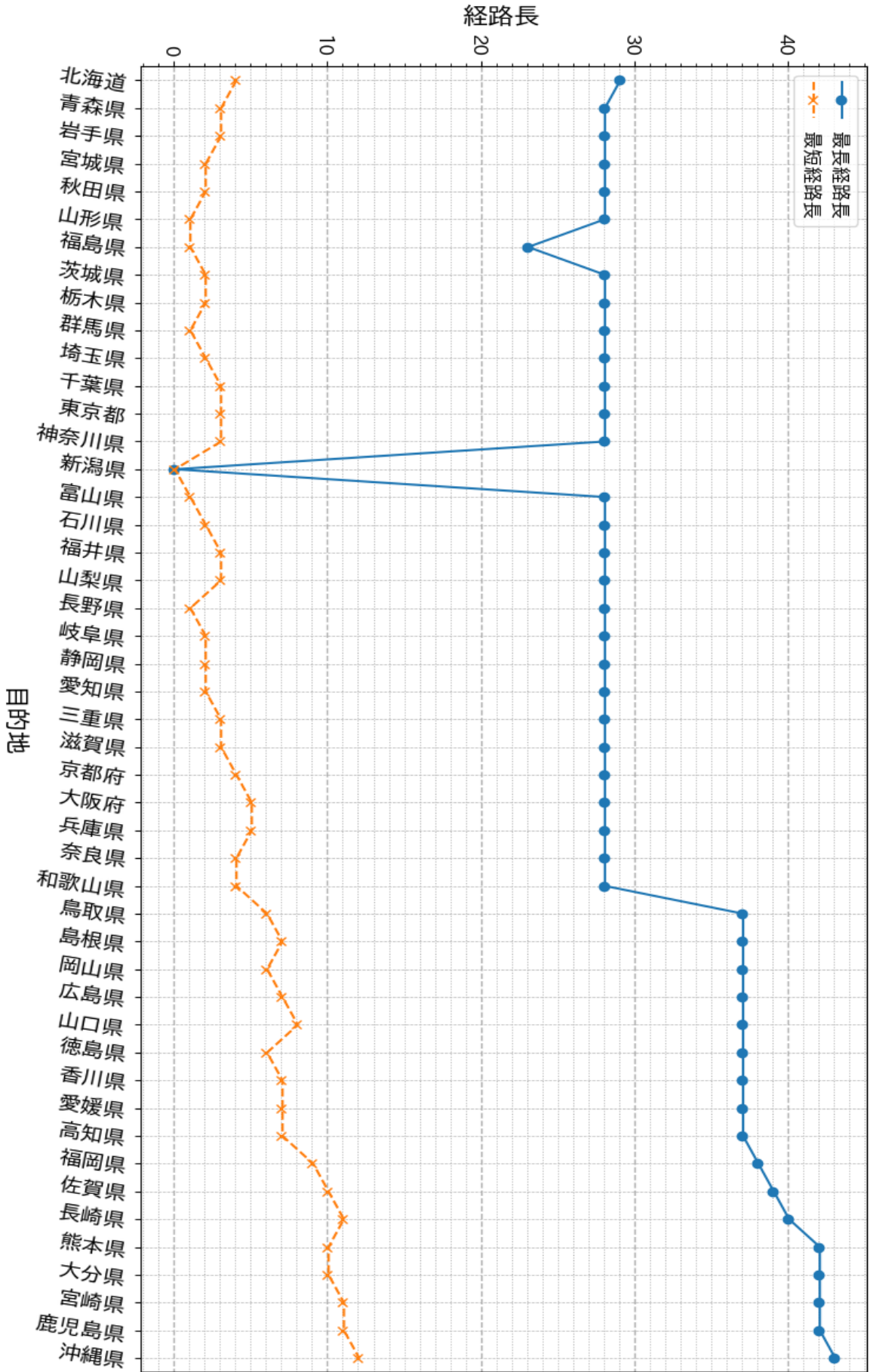
まず、各都道府県との最短経路長、最長経路長を示す。

本稿のタイトルは” 単一始点最長経路探索” としていたが、これまでのプログラムはユーザからの入力を受け付け終点の都道府県を指定していたため、2 頂点对最長経路探索ともいえる。このテストコードは全ての都道府県に対する最長経路探索を出力するため、確かに” 単一始点最長経路探索” のプログラムであるといえる。

ここでは、以下のライブラリを用いて python で結果をプロットした。

`matplotlib.pyplot as plt`、 `matplotlib.ticker as ticker`、 `japanize_matplotlib`

新潟県から各都道府県への最短・最長経路長



このような結果が得られた。私はここまでプログラムのテストには福島県と鹿児島県を多用していた。この二県を選択した理由はなんとなく短そうだから/長そうだからであった。この結果を見ると、47 種の目的地の内二番目に小さい値と二番目に大きい値の県をちょうどよく選択できていたとわかる。直観おそるべしといったところか。

また、最長経路が 28 と 37 に集中している点に興味深い。今後この要因を解析してみたい。

次に、全都道府県への最長経路を列挙する。

新潟へお越しの際は以下の実行結果をご活用いただきたい。

お住まいの都道府県の項目を参照し、矢印を逆にたどるだけで新潟県にたどり着く。

海外にお住まいの方は、東京都-新潟県間の経路なんかを参考にすればよいだろうか。

余白ができてしまったため、小言を一つばかり。

人生は山あり谷あり。しかし、焼きなまし法を思い返していただきたい。

苦しい道となるが、山に向かって進んでいけば、山を越えた先ではより良い景色に出会えるかもしれない。

なにも行動しなければ、見えている範囲の谷に落ち着くだけである。

まあ、その局所解も居心地の良い場所なのだろうが。

遠回りだって悪くない。

Graphillion を用いた都道府県最長経路探索（新潟県から各都道府県へ）

== 北海道 ==

29 県通過

経路：新潟県 -> 富山県 -> 石川県 -> 福井県 -> 滋賀県 -> 京都府 -> 兵庫県 -> 大阪府 -> 和歌山県 -> 奈良県 -> 三重県 -> 愛知県 -> 岐阜県 -> 長野県 -> 群馬県 -> 栃木県 -> 埼玉県 -> 山梨県 -> 静岡県 -> 神奈川県 -> 東京都 -> 千葉県 -> 茨城県 -> 福島県 -> 山形県 -> 宮城県 -> 岩手県 -> 秋田県 -> 青森県 -> 北海道

== 青森県 ==

28 県通過

経路：新潟県 -> 富山県 -> 石川県 -> 福井県 -> 滋賀県 -> 京都府 -> 兵庫県 -> 大阪府 -> 和歌山県 -> 奈良県 -> 三重県 -> 愛知県 -> 岐阜県 -> 長野県 -> 群馬県 -> 栃木県 -> 埼玉県 -> 山梨県 -> 静岡県 -> 神奈川県 -> 東京都 -> 千葉県 -> 茨城県 -> 福島県 -> 山形県 -> 宮城県 -> 岩手県 -> 秋田県 -> 青森県

== 岩手県 ==

28 県通過

経路：新潟県 -> 富山県 -> 石川県 -> 福井県 -> 滋賀県 -> 京都府 -> 兵庫県 -> 大阪府 -> 和歌山県 -> 奈良県 -> 三重県 -> 愛知県 -> 岐阜県 -> 長野県 -> 群馬県 -> 栃木県 -> 埼玉県 -> 山梨県 -> 静岡県 -> 神奈川県 -> 東京都 -> 千葉県 -> 茨城県 -> 福島県 -> 宮城県 -> 山形県 -> 秋田県 -> 青森県 -> 岩手県

== 宮城県 ==

28 県通過

経路：新潟県 -> 富山県 -> 石川県 -> 福井県 -> 滋賀県 -> 京都府 -> 兵庫県 -> 大阪府 -> 和歌山県 -> 奈良県 -> 三重県 -> 愛知県 -> 岐阜県 -> 長野県 -> 群馬県 -> 栃木県 -> 埼玉県 -> 山梨県 -> 静岡県 -> 神奈川県 -> 東京都 -> 千葉県 -> 茨城県 -> 福島県 -> 山形県 -> 秋田県 -> 青森県 -> 岩手県 -> 宮城県

== 秋田県 ==

28 県通過

経路：新潟県 -> 富山県 -> 石川県 -> 福井県 -> 滋賀県 -> 京都府 -> 兵庫県 -> 大阪府 -> 和歌山県 -> 奈良県 -> 三重県 -> 愛知県 -> 岐阜県 -> 長野県 -> 群馬県 -> 栃木県 -> 埼玉県 -> 山梨県 -> 静岡県 -> 神奈川県 -> 東京都 -> 千葉県 -> 茨城県 -> 福島県 -> 山形県 -> 宮城県 -> 岩手県 -> 青森県 -> 秋田県

== 山形県 ==

28 県通過

経路：新潟県 -> 富山県 -> 石川県 -> 福井県 -> 滋賀県 -> 京都府 -> 兵庫県 -> 大阪府 -> 和歌山県 -> 奈良県 -> 三重県 -> 愛知県 -> 岐阜県 -> 長野県 -> 群馬県 -> 栃木県 -> 埼玉県 -> 山梨県 -> 静岡県 -> 神奈川県 -> 東京都 -> 千葉県 -> 茨城県 -> 福島県 -> 宮城県 -> 岩手県 -> 青森県 -> 秋田県 -> 山形県

== 福島県 ==

23 県通過

経路：新潟県 -> 富山県 -> 石川県 -> 福井県 -> 滋賀県 -> 京都府 -> 兵庫県 -> 大阪府 -> 和歌山県 -> 奈良県 -> 三重県 -> 愛知県 -> 岐阜県 -> 長野県 -> 群馬県 -> 栃木県 -> 埼玉県 -> 山梨県 -> 静岡県 -> 神奈川県 -> 東京都 -> 千葉県 -> 茨城県 -> 福島県

== 茨城県 ==

28 県通過

経路：新潟県 -> 山形県 -> 秋田県 -> 青森県 -> 岩手県 -> 宮城県 -> 福島県 -> 栃木県 -> 埼玉県 -> 群馬県 -> 長野県 -> 岐阜県 -> 富山県 -> 石川県 -> 福井県 -> 滋賀県 -> 京都府 -> 兵庫県 -> 大阪府 -> 和歌山県 -> 奈良県 -> 三重県 -> 愛知県 -> 静岡県 -> 山梨県 -> 神奈川県 -> 東京都 -> 千葉県 -> 茨城県

== 栃木県 ==

28 県通過

経路：新潟県 -> 山形県 -> 秋田県 -> 青森県 -> 岩手県 -> 宮城県 -> 福島県 -> 茨城県 -> 千葉県 -> 東京都 -> 神奈川県 -> 山梨県 -> 静岡県 -> 愛知県 -> 三重県 -> 奈良県 -> 和歌山県 -> 大阪府 -> 兵庫県 -> 京都府 -> 滋賀県 -> 福井県 -> 石川県 -> 富山県 -> 岐阜県 -> 長野県 -> 群馬県 -> 埼玉県 -> 栃木県

== 群馬県 ==

28 県通過

経路：新潟県 -> 山形県 -> 秋田県 -> 青森県 -> 岩手県 -> 宮城県 -> 福島県 -> 茨城県 -> 栃木県 -> 埼玉県 -> 千葉県 -> 東京都 -> 神奈川県 -> 山梨県 -> 静岡県 -> 愛知県 -> 三重県 -> 奈良県 -> 和歌山県 -> 大阪府 -> 兵庫県 -> 京都府 -> 滋賀県 -> 福井県 -> 石川県 -> 富山県 -> 岐阜県 -> 長野県 -> 群馬県

== 埼玉県 ==

28 県通過

経路：新潟県 -> 山形県 -> 秋田県 -> 青森県 -> 岩手県 -> 宮城県 -> 福島県 -> 茨城県 -> 千葉県 -> 東京都 -> 神奈川県 -> 山梨県 -> 静岡県 -> 愛知県 -> 三重県 -> 奈良県 -> 和歌山県 -> 大阪府 -> 兵庫県 -> 京都府 -> 滋賀県 -> 福井県 -> 石川県 -> 富山県 -> 岐阜県 -> 長野県 -> 群馬県 -> 栃木県 -> 埼玉県

== 千葉県 ==

28 県通過

経路：新潟県 -> 山形県 -> 秋田県 -> 青森県 -> 岩手県 -> 宮城県 -> 福島県 -> 茨城県 -> 埼玉県 -> 栃木県 -> 群馬県 -> 長野県 -> 岐阜県 -> 富山県 -> 石川県 -> 福井県 -> 滋賀県 -> 京都府 -> 兵庫県 -> 大阪府 -> 和歌山県 -> 奈良県 -> 三重県 -> 愛知県 -> 静岡県 -> 山梨県 -> 神奈川県 -> 東京都 -> 千葉県

== 東京都 ==

28 県通過

経路：新潟県 -> 山形県 -> 秋田県 -> 青森県 -> 岩手県 -> 宮城県 -> 福島県 -> 茨城県 -> 千葉県 -> 埼玉県 -> 栃木県 -> 群馬県 -> 長野県 -> 岐阜県 -> 富山県 -> 石川県 -> 福井県 -> 滋賀県 -> 京都府 -> 兵庫県 -> 大阪府 -> 和歌山県 -> 奈良県 -> 三重県 -> 愛知県 -> 静岡県 -> 山梨県 -> 神奈川県 -> 東京都

== 神奈川県 ==

28 県通過

経路：新潟県 -> 山形県 -> 秋田県 -> 青森県 -> 岩手県 -> 宮城県 -> 福島県 -> 茨城県 -> 千葉県 -> 東京都 -> 山梨県 -> 埼玉県 -> 栃木県 -> 群馬県 -> 長野県 -> 岐阜県 -> 富山県 -> 石川県 -> 福井県 -> 滋賀県 -> 京都府 -> 兵庫県 -> 大阪府 -> 和歌山県 -> 奈良県 -> 三重県 -> 愛知県 -> 静岡県 -> 神奈川県

== 新潟県 ==

出発地と目的地が同じです。

== 富山県 ==

28 県通過

経路：新潟県 -> 山形県 -> 秋田県 -> 青森県 -> 岩手県 -> 宮城県 -> 福島県 -> 茨城県 -> 千葉県 -> 東京都 -> 神奈川県 -> 静岡県 -> 山梨県 -> 埼玉県 -> 栃木県 -> 群馬県 -> 長野県 -> 岐阜県 -> 愛知県 -> 三重県 -> 奈良県 -> 和歌山県 -> 大阪府 -> 兵庫県 -> 京都府 -> 滋賀県 -> 福井県 -> 石川県 -> 富山県

== 石川県 ==

28 県通過

経路：新潟県 -> 山形県 -> 秋田県 -> 青森県 -> 岩手県 -> 宮城県 -> 福島県 -> 茨城県 -> 千葉県 -> 東京都 -> 神奈川県 -> 静岡県 -> 山梨県 -> 埼玉県 -> 栃木県 -> 群馬県 -> 長野県 -> 富山県 -> 岐阜県 -> 愛知県 -> 三重県 -> 奈良県 -> 和歌山県 -> 大阪府 -> 兵庫県 -> 京都府 -> 滋賀県 -> 福井県 -> 石川県

== 福井県 ==

28 県通過

経路：新潟県 -> 山形県 -> 秋田県 -> 青森県 -> 岩手県 -> 宮城県 -> 福島県 -> 茨城県 -> 千葉県 -> 東京都 -> 神奈川県 -> 静岡県 -> 山梨県 -> 埼玉県 -> 栃木県 -> 群馬県 -> 長野県 -> 富山県 -> 石川県 -> 岐阜県 -> 愛知県 -> 三重県 -> 奈良県 -> 和歌山県 -> 大阪府 -> 兵庫県 -> 京都府 -> 滋賀県 -> 福井県

== 山梨県 ==

28 県通過

経路：新潟県 -> 山形県 -> 秋田県 -> 青森県 -> 岩手県 -> 宮城県 -> 福島県 -> 茨城県 -> 千葉県 -> 東京都 -> 神奈川県 -> 静岡県 -> 愛知県 -> 三重県 -> 奈良県 -> 和歌山県 -> 大阪府 -> 兵庫県 -> 京都府 -> 滋賀県 -> 福井県 -> 石川県 -> 富山県 -> 岐阜県 -> 長野県 -> 群馬県 -> 栃木県 -> 埼玉県 -> 山梨県

== 長野県 ==

28 県通過

経路：新潟県 -> 山形県 -> 秋田県 -> 青森県 -> 岩手県 -> 宮城県 -> 福島県 -> 茨城県 -> 群馬県 -> 栃木県 -> 埼玉県 -> 千葉県 -> 東京都 -> 神奈川県 -> 山梨県 -> 静岡県 -> 愛知県 -> 三重県 -> 奈良県 -> 和歌山県 -> 大阪府 -> 兵庫県 -> 京都府 -> 滋賀県 -> 福井県 -> 石川県 -> 富山県 -> 岐阜県 -> 長野県

== 岐阜県 ==

28 県通過

経路：新潟県 -> 山形県 -> 秋田県 -> 青森県 -> 岩手県 -> 宮城県 -> 福島県 -> 茨城県 -> 千葉県 -> 東京都 -> 神奈川県 -> 静岡県 -> 山梨県 -> 埼玉県 -> 栃木県 -> 群馬県 -> 長野県 -> 富山県 -> 石川県 -> 福井県 -> 滋賀県 -> 京都府 -> 兵庫県 -> 大阪府 -> 和歌山県 -> 奈良県 -> 三重県 -> 愛知県 -> 岐阜県

== 静岡県 ==

28 県通過

経路：新潟県 -> 山形県 -> 秋田県 -> 青森県 -> 岩手県 -> 宮城県 -> 福島県 -> 茨城県 -> 千葉県 -> 東京都 -> 神奈川県 -> 山梨県 -> 埼玉県 -> 栃木県 -> 群馬県 -> 長野県 -> 岐阜県 -> 富山県 -> 石川県 -> 福井県 -> 滋賀県 -> 京都府 -> 兵庫県 -> 大阪府 -> 和歌山県 -> 奈良県 -> 三重県 -> 愛知県 -> 静岡県

== 愛知県 ==

28 県通過

経路：新潟県 -> 山形県 -> 秋田県 -> 青森県 -> 岩手県 -> 宮城県 -> 福島県 -> 茨城県 -> 千葉県 -> 東京都 -> 神奈川県 -> 静岡県 -> 山梨県 -> 埼玉県 -> 栃木県 -> 群馬県 -> 長野県 -> 岐阜県 -> 富山県 -> 石川県 -> 福井県 -> 滋賀県 -> 京都府 -> 兵庫県 -> 大阪府 -> 和歌山県 -> 奈良県 -> 三重県 -> 愛知県

== 三重県 ==

28 県通過

経路：新潟県 -> 山形県 -> 秋田県 -> 青森県 -> 岩手県 -> 宮城県 -> 福島県 -> 茨城県 -> 千葉県 -> 東京都 -> 神奈川県 -> 静岡県 -> 山梨県 -> 埼玉県 -> 栃木県 -> 群馬県 -> 長野県 -> 愛知県 -> 岐阜県 -> 富山県 -> 石川県 -> 福井県 -> 滋賀県 -> 京都府 -> 兵庫県 -> 大阪府 -> 和歌山県 -> 奈良県 -> 三重県

== 滋賀県 ==

28 県通過

経路：新潟県 -> 山形県 -> 秋田県 -> 青森県 -> 岩手県 -> 宮城県 -> 福島県 -> 茨城県 -> 千葉県 -> 東京都 -> 神奈川県 -> 静岡県 -> 山梨県 -> 埼玉県 -> 栃木県 -> 群馬県 -> 長野県 -> 富山県 -> 石川県 -> 福井県 -> 京都府 -> 兵庫県 -> 大阪府 -> 和歌山県 -> 奈良県 -> 三重県 -> 愛知県 -> 岐阜県 -> 滋賀県

== 京都府 ==

28 県通過

経路：新潟県 -> 山形県 -> 秋田県 -> 青森県 -> 岩手県 -> 宮城県 -> 福島県 -> 茨城県 -> 千葉県 -> 東京都 -> 神奈川県 -> 静岡県 -> 山梨県 -> 埼玉県 -> 栃木県 -> 群馬県 -> 長野県 -> 富山県 -> 石川県 -> 福井県 -> 滋賀県 -> 岐阜県 -> 愛知県 -> 三重県 -> 奈良県 -> 和歌山県 -> 大阪府 -> 兵庫県 -> 京都府

== 大阪府 ==

28 県通過

経路：新潟県 -> 山形県 -> 秋田県 -> 青森県 -> 岩手県 -> 宮城県 -> 福島県 -> 茨城県 -> 千葉県 -> 東京都 -> 神奈川県 -> 静岡県 -> 山梨県 -> 埼玉県 -> 栃木県 -> 群馬県 -> 長野県 -> 富山県 -> 石川県 -> 福井県 -> 滋賀県 -> 岐阜県 -> 愛知県 -> 三重県 -> 和歌山県 -> 奈良県 -> 京都府 -> 兵庫県 -> 大阪府

== 兵庫県 ==

28 県通過

経路：新潟県 -> 山形県 -> 秋田県 -> 青森県 -> 岩手県 -> 宮城県 -> 福島県 -> 茨城県 -> 千葉県 -> 東京都 -> 神奈川県 -> 静岡県 -> 山梨県 -> 埼玉県 -> 栃木県 -> 群馬県 -> 長野県 -> 富山県 -> 石川県 -> 福井県 -> 京都府 -> 滋賀県 -> 岐阜県 -> 愛知県 -> 三重県 -> 奈良県 -> 和歌山県 -> 大阪府 -> 兵庫県

== 奈良県 ==

28 県通過

経路：新潟県 -> 山形県 -> 秋田県 -> 青森県 -> 岩手県 -> 宮城県 -> 福島県 -> 茨城県 -> 千葉県 -> 東京都 -> 神奈川県 -> 静岡県 -> 山梨県 -> 埼玉県 -> 栃木県 -> 群馬県 -> 長野県 -> 富山県 -> 石川県 -> 福井県 -> 滋賀県 -> 岐阜県 -> 愛知県 -> 三重県 -> 和歌山県 -> 大阪府 -> 兵庫県 -> 京都府 -> 奈良県

== 和歌山県 ==

28 県通過

経路：新潟県 -> 山形県 -> 秋田県 -> 青森県 -> 岩手県 -> 宮城県 -> 福島県 -> 茨城県 -> 千葉県 -> 東京都 -> 神奈川県 -> 静岡県 -> 山梨県 -> 埼玉県 -> 栃木県 -> 群馬県 -> 長野県 -> 富山県 -> 石川県 -> 福井県 -> 滋賀県 -> 岐阜県 -> 愛知県 -> 三重県 -> 奈良県 -> 京都府 -> 兵庫県 -> 大阪府 -> 和歌山県

== 鳥取県 ==

37 県通過

経路：新潟県 -> 山形県 -> 秋田県 -> 青森県 -> 岩手県 -> 宮城県 -> 福島県 -> 茨城県 -> 千葉県 -> 東京都 -> 神奈川県 -> 静岡県 -> 山梨県 -> 埼玉県 -> 栃木県 -> 群馬県 -> 長野県 -> 富山県 -> 石川県 -> 福井県 -> 京都府 -> 滋賀県 -> 岐阜県 -> 愛知県 -> 三重県 -> 奈良県 -> 和歌山県 -> 大阪府 -> 兵庫県 -> 徳島県 -> 高知県 -> 愛媛県 -> 香川県 -> 岡山県 -> 広島県 -> 山口県 -> 島根県 -> 鳥取県

== 島根県 ==

37 県通過

経路：新潟県 -> 山形県 -> 秋田県 -> 青森県 -> 岩手県 -> 宮城県 -> 福島県 -> 茨城県 -> 千葉県 -> 東京都 -> 神奈川県 -> 静岡県 -> 山梨県 -> 埼玉県 -> 栃木県 -> 群馬県 -> 長野県 -> 富山県 -> 石川県 -> 福井県 -> 京都府 -> 滋賀県 -> 岐阜県 -> 愛知県 -> 三重県 -> 奈良県 -> 和歌山県 -> 大阪府 -> 兵庫県 -> 徳島県 -> 高知県 -> 愛媛県 -> 香川県 -> 岡山県 -> 鳥取県 -> 広島県 -> 山口県 -> 島根県

== 岡山県 ==

37 県通過

経路：新潟県 -> 山形県 -> 秋田県 -> 青森県 -> 岩手県 -> 宮城県 -> 福島県 -> 茨城県 -> 千葉県 -> 東京都 -> 神奈川県 -> 静岡県 -> 山梨県 -> 埼玉県 -> 栃木県 -> 群馬県 -> 長野県 -> 富山県 -> 石川県 -> 福井県 -> 京都府 -> 滋賀県 -> 岐阜県 -> 愛知県 -> 三重県 -> 奈良県 -> 和歌山県 -> 大阪府 -> 兵庫県 -> 鳥取県 -> 島根県 -> 山口県 -> 広島県 -> 愛媛県 -> 高知県 -> 徳島県 -> 香川県 -> 岡山県

== 広島県 ==

37 県通過

経路：新潟県 -> 山形県 -> 秋田県 -> 青森県 -> 岩手県 -> 宮城県 -> 福島県 -> 茨城県 -> 千葉県 -> 東京都 -> 神奈川県 -> 静岡県 -> 山梨県 -> 埼玉県 -> 栃木県 -> 群馬県 -> 長野県 -> 富山県 -> 石川県 -> 福井県 -> 京都府 -> 滋賀県 -> 岐阜県 -> 愛知県 -> 三重県 -> 奈良県 -> 和歌山県 -> 大阪府 -> 兵庫県 -> 徳島県 -> 高知県 -> 愛媛県 -> 香川県 -> 岡山県 -> 鳥取県 -> 島根県 -> 山口県 -> 広島県

== 山口県 ==

37 県通過

経路：新潟県 -> 山形県 -> 秋田県 -> 青森県 -> 岩手県 -> 宮城県 -> 福島県 -> 茨城県 -> 千葉県 -> 東京都 -> 神奈川県 -> 静岡県 -> 山梨県 -> 埼玉県 -> 栃木県 -> 群馬県 -> 長野県 -> 富山県 -> 石川県 -> 福井県 -> 京都府 -> 滋賀県 -> 岐阜県 -> 愛知県 -> 三重県 -> 奈良県 -> 和歌山県 -> 大阪府 -> 兵庫県 -> 徳島県 -> 高知県 -> 愛媛県 -> 香川県 -> 岡山県 -> 広島県 -> 鳥取県 -> 島根県 -> 山口県

== 徳島県 ==

37 県通過

経路：新潟県 -> 山形県 -> 秋田県 -> 青森県 -> 岩手県 -> 宮城県 -> 福島県 -> 茨城県
-> 千葉県 -> 東京都 -> 神奈川県 -> 静岡県 -> 山梨県 -> 埼玉県 -> 栃木県 -> 群馬県
-> 長野県 -> 富山県 -> 石川県 -> 福井県 -> 京都府 -> 滋賀県 -> 岐阜県 -> 愛知県 ->
三重県 -> 奈良県 -> 和歌山県 -> 大阪府 -> 兵庫県 -> 鳥取県 -> 島根県 -> 山口県 ->
広島県 -> 岡山県 -> 香川県 -> 愛媛県 -> 高知県 -> 徳島県

== 香川県 ==

37 県通過

経路：新潟県 -> 山形県 -> 秋田県 -> 青森県 -> 岩手県 -> 宮城県 -> 福島県 -> 茨城県
-> 千葉県 -> 東京都 -> 神奈川県 -> 静岡県 -> 山梨県 -> 埼玉県 -> 栃木県 -> 群馬県
-> 長野県 -> 富山県 -> 石川県 -> 福井県 -> 京都府 -> 滋賀県 -> 岐阜県 -> 愛知県 ->
三重県 -> 奈良県 -> 和歌山県 -> 大阪府 -> 兵庫県 -> 徳島県 -> 高知県 -> 愛媛県 ->
広島県 -> 山口県 -> 島根県 -> 鳥取県 -> 岡山県 -> 香川県

== 愛媛県 ==

37 県通過

経路：新潟県 -> 山形県 -> 秋田県 -> 青森県 -> 岩手県 -> 宮城県 -> 福島県 -> 茨城県
-> 千葉県 -> 東京都 -> 神奈川県 -> 静岡県 -> 山梨県 -> 埼玉県 -> 栃木県 -> 群馬県
-> 長野県 -> 富山県 -> 石川県 -> 福井県 -> 京都府 -> 滋賀県 -> 岐阜県 -> 愛知県 ->
三重県 -> 奈良県 -> 和歌山県 -> 大阪府 -> 兵庫県 -> 鳥取県 -> 島根県 -> 山口県 ->
広島県 -> 岡山県 -> 香川県 -> 徳島県 -> 高知県 -> 愛媛県

== 高知県 ==

37 県通過

経路：新潟県 -> 山形県 -> 秋田県 -> 青森県 -> 岩手県 -> 宮城県 -> 福島県 -> 茨城県
-> 千葉県 -> 東京都 -> 神奈川県 -> 静岡県 -> 山梨県 -> 埼玉県 -> 栃木県 -> 群馬県
-> 長野県 -> 富山県 -> 石川県 -> 福井県 -> 京都府 -> 滋賀県 -> 岐阜県 -> 愛知県 ->
三重県 -> 奈良県 -> 和歌山県 -> 大阪府 -> 兵庫県 -> 徳島県 -> 香川県 -> 岡山県 ->
鳥取県 -> 島根県 -> 山口県 -> 広島県 -> 愛媛県 -> 高知県

== 福岡県 ==

38 県通過

経路：新潟県 -> 山形県 -> 秋田県 -> 青森県 -> 岩手県 -> 宮城県 -> 福島県 -> 茨城県
-> 千葉県 -> 東京都 -> 神奈川県 -> 静岡県 -> 山梨県 -> 埼玉県 -> 栃木県 -> 群馬県
-> 長野県 -> 富山県 -> 石川県 -> 福井県 -> 京都府 -> 滋賀県 -> 岐阜県 -> 愛知県 ->
三重県 -> 奈良県 -> 和歌山県 -> 大阪府 -> 兵庫県 -> 徳島県 -> 高知県 -> 愛媛県 ->
香川県 -> 岡山県 -> 広島県 -> 鳥取県 -> 島根県 -> 山口県 -> 福岡県

== 佐賀県 ==

39 県通過

経路：新潟県 -> 山形県 -> 秋田県 -> 青森県 -> 岩手県 -> 宮城県 -> 福島県 -> 茨城県
-> 千葉県 -> 東京都 -> 神奈川県 -> 静岡県 -> 山梨県 -> 埼玉県 -> 栃木県 -> 群馬県
-> 長野県 -> 富山県 -> 石川県 -> 福井県 -> 京都府 -> 滋賀県 -> 岐阜県 -> 愛知県 ->
三重県 -> 奈良県 -> 和歌山県 -> 大阪府 -> 兵庫県 -> 徳島県 -> 高知県 -> 愛媛県 ->
香川県 -> 岡山県 -> 広島県 -> 鳥取県 -> 島根県 -> 山口県 -> 福岡県 -> 佐賀県

== 長崎県 ==

40 県通過

経路：新潟県 -> 山形県 -> 秋田県 -> 青森県 -> 岩手県 -> 宮城県 -> 福島県 -> 茨城県 -> 千葉県 -> 東京都 -> 神奈川県 -> 静岡県 -> 山梨県 -> 埼玉県 -> 栃木県 -> 群馬県 -> 長野県 -> 富山県 -> 石川県 -> 福井県 -> 京都府 -> 滋賀県 -> 岐阜県 -> 愛知県 -> 三重県 -> 奈良県 -> 和歌山県 -> 大阪府 -> 兵庫県 -> 徳島県 -> 高知県 -> 愛媛県 -> 香川県 -> 岡山県 -> 広島県 -> 鳥取県 -> 島根県 -> 山口県 -> 福岡県 -> 佐賀県 -> 長崎県

== 熊本県 ==

42 県通過

経路：新潟県 -> 山形県 -> 秋田県 -> 青森県 -> 岩手県 -> 宮城県 -> 福島県 -> 茨城県 -> 千葉県 -> 東京都 -> 神奈川県 -> 静岡県 -> 山梨県 -> 埼玉県 -> 栃木県 -> 群馬県 -> 長野県 -> 富山県 -> 石川県 -> 福井県 -> 京都府 -> 滋賀県 -> 岐阜県 -> 愛知県 -> 三重県 -> 奈良県 -> 和歌山県 -> 大阪府 -> 兵庫県 -> 徳島県 -> 高知県 -> 愛媛県 -> 香川県 -> 岡山県 -> 広島県 -> 鳥取県 -> 島根県 -> 山口県 -> 福岡県 -> 大分県 -> 宮崎県 -> 鹿児島県 -> 熊本県

== 大分県 ==

42 県通過

経路：新潟県 -> 山形県 -> 秋田県 -> 青森県 -> 岩手県 -> 宮城県 -> 福島県 -> 茨城県 -> 千葉県 -> 東京都 -> 神奈川県 -> 静岡県 -> 山梨県 -> 埼玉県 -> 栃木県 -> 群馬県 -> 長野県 -> 富山県 -> 石川県 -> 福井県 -> 京都府 -> 滋賀県 -> 岐阜県 -> 愛知県 -> 三重県 -> 奈良県 -> 和歌山県 -> 大阪府 -> 兵庫県 -> 徳島県 -> 高知県 -> 愛媛県 -> 香川県 -> 岡山県 -> 広島県 -> 鳥取県 -> 島根県 -> 山口県 -> 福岡県 -> 熊本県 -> 鹿児島県 -> 宮崎県 -> 大分県

== 宮崎県 ==

42 県通過

経路：新潟県 -> 山形県 -> 秋田県 -> 青森県 -> 岩手県 -> 宮城県 -> 福島県 -> 茨城県 -> 千葉県 -> 東京都 -> 神奈川県 -> 静岡県 -> 山梨県 -> 埼玉県 -> 栃木県 -> 群馬県 -> 長野県 -> 富山県 -> 石川県 -> 福井県 -> 京都府 -> 滋賀県 -> 岐阜県 -> 愛知県 -> 三重県 -> 奈良県 -> 和歌山県 -> 大阪府 -> 兵庫県 -> 徳島県 -> 高知県 -> 愛媛県 -> 香川県 -> 岡山県 -> 広島県 -> 鳥取県 -> 島根県 -> 山口県 -> 福岡県 -> 大分県 -> 熊本県 -> 鹿児島県 -> 宮崎県

== 鹿児島県 ==

42 県通過

経路：新潟県 -> 山形県 -> 秋田県 -> 青森県 -> 岩手県 -> 宮城県 -> 福島県 -> 茨城県 -> 千葉県 -> 東京都 -> 神奈川県 -> 静岡県 -> 山梨県 -> 埼玉県 -> 栃木県 -> 群馬県 -> 長野県 -> 富山県 -> 石川県 -> 福井県 -> 京都府 -> 滋賀県 -> 岐阜県 -> 愛知県 -> 三重県 -> 奈良県 -> 和歌山県 -> 大阪府 -> 兵庫県 -> 徳島県 -> 高知県 -> 愛媛県 -> 香川県 -> 岡山県 -> 広島県 -> 鳥取県 -> 島根県 -> 山口県 -> 福岡県 -> 熊本県 -> 大分県 -> 宮崎県 -> 鹿児島県

== 沖縄県 ==

43 県通過

経路：新潟県 -> 山形県 -> 秋田県 -> 青森県 -> 岩手県 -> 宮城県 -> 福島県 -> 茨城県 -> 千葉県 -> 東京都 -> 神奈川県 -> 静岡県 -> 山梨県 -> 埼玉県 -> 栃木県 -> 群馬県 -> 長野県 -> 富山県 -> 石川県 -> 福井県 -> 京都府 -> 滋賀県 -> 岐阜県 -> 愛知県 -> 三重県 -> 奈良県 -> 和歌山県 -> 大阪府 -> 兵庫県 -> 徳島県 -> 高知県 -> 愛媛県 -> 香川県 -> 岡山県 -> 広島県 -> 鳥取県 -> 島根県 -> 山口県 -> 福岡県 -> 熊本県 -> 大分県 -> 宮崎県 -> 鹿児島県 -> 沖縄県

謝辞

本稿の執筆及びプログラムの作成にあたり協力してくれた友人、応援してくれた同僚に最大限の感謝を。

参考文献の執筆やライブラリの実装に関わった先駆者の皆様には最大限の敬意を。

本稿を参考に最長経路で新潟へお越しになる方へ最大限のエールを。

また本稿では、プログラムの作成とテストコードの作成、QUBO 行列の作成にあたり生成 AI の補助を受けた。

皆様の長旅欲、あるいは経路探索欲を刺激できたなら嬉しい。

参考文献

- [1] 都道府県市区町村 | 都道府県隣接関係一覧
<https://uub.jp/prf/rinsetsu.html>
- [2] ゆるコンピュータ科学ラジオ | Google マップはなぜ爆速で最短経路が分かるの？
<https://www.youtube.com/watch?v=n5bnIYI2t78>
- [3] アルゴリズムイントロダクション[第一巻] 第4版 | 近代科学社
T. コルメン, C. ライザーソン, R. リベスト, C. シュタイン 共著
浅野哲夫, 岩野和生, 梅尾博司, 小山透, 山下雅史, 和田幸一 共訳
- [4] 新潟大学コンピュータクラブ | 2024 年度年刊 EDP
三日坊主は量子計算の夢を見られるのか | 遠雷亭
<https://edp.mods.jp/work.html>

[5] Github | Graphillion

<https://github.com/takemaru/graphillion>

[6] Graphillion で東京メトロの最長経路問題を考える

<https://tech.mobilefactory.jp/entry/2018/12/20/180000>

[7] 超高速グラフ列挙アルゴリズム:〈フカシギの数え方〉が拓く,組合せ問題への新アプローチ | 森北出版 | 湊真一 著

[8] BDD と ZDD -ブール関数と集合族の圧縮処理-

https://qiita.com/tsukasa_diary/items/d2b5940dacf3c3a8d3ff

Github:重み無しグラフ単一始点最長経路探索

https://github.com/HondaEnlightea/longest_pathfinder